

# WERTHAN DAO Project Review

Date: 2026-08-12 | Branch reviewed: dev | Scope: 45-60 minute DB-less review, local run, frontend/backend walkthrough, Web3 flow review, and testnet readiness check.

## Project Understanding

WERTHAN DAO is a Next.js Web3 application for a DAO-style ecosystem. The product combines a public marketing site, wallet connection, proposal creation, voting, escrow/treasury visibility, reward release flows, token information, and MongoDB-backed proposal metadata. The intended architecture is hybrid: smart contracts handle governance and rewards on-chain, while MongoDB stores proposal metadata, files, and sync status off-chain.

## 1. Issues Encountered While Running The Project

- The project installed and ran locally, but the native Next.js SWC package failed on this Windows/Node environment. Installing the matching WASM SWC package allowed `npm run dev` to start.
- The main pages were reviewable without MongoDB: `/`, `/vote`, and `/reward` rendered successfully. `/vote` and `/reward` correctly gate core actions behind wallet connection.
- `/api/health` returned successfully and reported Mongo as disconnected, which matches the DB-less review scope.
- Proposal API routes such as `/api/proposals/live`, `/api/proposals/pending`, and `/api/proposals/past` hang when MongoDB is unavailable because their error handlers log failures but do not return an HTTP response.
- `npm run build` is fragile because `pages/index.js` imports `Inter` from `next/font/google`. In this environment, the Google Fonts fetch timed out, even though the imported font constant appears unused.
- The README setup commands do not fully match `package.json`. For example, the repo documents commands like `npm dev` and `npm build`, but the actual commands are `npm run dev`, `npm run build`, and `npm run lint`.

## 2. Top 3 Technical Risks

1. **High - Core governance voting is incomplete.** The approve and deny handlers in the vote page are empty, and the Web3 vote helper is commented out. This means users can view governance screens, but the central DAO action - casting votes - is not currently functional from the UI.
2. **High - Network configuration is not safely testnet-ready.** The `.env` file contains testnet-style values, but the running app imports hardcoded Polygon mainnet chain and contract constants from `utils/_constants.js`. This makes it easy to think the app is using testnet while it is actually pointed at mainnet configuration.
3. **Medium/High - On-chain/off-chain sync can drift.** Proposal creation saves a MongoDB record before the wallet transaction is completed, then syncs the on-chain proposal id afterward. If the wallet transaction is rejected or fails, Mongo can contain orphan or unsynced proposals. The same area also needs stronger API error responses and recovery logic.

## 3. MVP Priorities

- **Make environment and testnet configuration reliable first.** Move chain id, chain name, RPC URL, and contract addresses into environment-driven config, then provide a known working testnet deployment.
- **Finish the proposal lifecycle.** Implement approve/deny voting, restore the Web3 vote helper, verify proposal creation, and add sync/retry behavior for failed wallet transactions.
- **Fix correctness issues in Web3 and dashboard data.** Align proposal amount units between gas estimation and transaction send, fix escrow balance state mapping, and correct the FTO balance helper so it reads the intended contract.
- **Harden user-generated content and APIs.** Sanitize proposal descriptions rendered with `dangerouslySetInnerHTML`, and ensure every API route returns clear JSON on failure instead of hanging.
- **Add a small smoke-test suite.** Cover DB-less rendering, API failure behavior, and at least one mocked governance flow so future reviewers can validate the app quickly.

## 4. Developer Experience Improvements

- Update README setup commands and add a clear "DB-less review mode" section.
- Add a complete `.env.example` with separate sections for local MongoDB, testnet RPC, network id, contract addresses, and public-only browser variables.
- Document which contracts are deployed on which network, and match each address to its ABI.
- Add sample proposal data or a mock proposal mode so reviewers can inspect proposal states without needing MongoDB.
- Make linting non-interactive by committing an ESLint configuration.
- Add health checks for MongoDB, RPC connectivity, chain id, and contract availability.
- Document the expected review flow: start app, connect test wallet, create proposal, vote, verify Mongo sync, inspect rewards.

## 5. 30-Day Execution Plan

**Week 1 - Setup, config, and build reliability.** Fix README commands, add `.env.example`, remove or self-host the unused Google Font dependency, make network/contract config environment-driven, and confirm a working testnet RPC plus deployed testnet addresses.

**Week 2 - Governance MVP.** Complete proposal creation and approve/deny voting from the UI, fix amount unit handling, add transaction status states, and implement sync recovery for failed or rejected wallet transactions.

**Week 3 - Rewards, escrow, and API hardening.** Fix reward eligibility case handling, wire any static reward buttons, correct escrow balance displays, sanitize proposal HTML, and make all API routes return structured error responses.

**Week 4 - End-to-end validation and release readiness.** Run the full testnet walkthrough with a funded test wallet, validate on-chain events against off-chain Mongo records, add smoke/E2E tests, prepare deployment notes, and create a reviewer checklist for future handoffs.

## 6. Candid Opinion: Technical Viability And Excitement

The project is technically viable. The architecture is recognizable and reasonable for this type of DAO product: Next.js frontend, wallet connectors, Web3 contract reads/writes, MongoDB proposal metadata, treasury views, and reward release flows.

My excitement level is positive because the product direction is concrete and the main pieces are already present. My caution is that it is not MVP-ready yet. The biggest blockers are practical execution issues: voting is incomplete, testnet/mainnet separation is unclear, proposal sync can drift, and API failures are not handled cleanly.

If the team focuses the next sprint on testnet configuration, complete governance actions, and reliable proposal sync, this can become a much stronger hands-on MVP quickly.